

Final Report – Group 1

Social Media Platform – *Shout!*

Auri Laitinen

Niko Mehiläinen

Samuel Sarimo

Table of Contents

Final Report – Group 1	1
Social Media Platform – <i>Shout!</i>	1
Part I: Project Development Process	2
1. Applied Methodology	2
2. Planning and Risk Analysis	2
3. Agile Implementation Approach	2
4. Scrum Team Roles and Responsibilities	2
Project Links:	2
Roles:	2
Part II: Product Documentation	3
1. Introduction	3
2. Design	3
2.1 ER Diagrams	3
2.2 UML Diagrams	4
2.3 UI Mockups	5
3. Features Overview	6
4. Implementation Details	6
5. Testing Strategy and Results	7
5.1 Testing Strategy	7
5.2 Test Results	8
6. Installation Guide	9
7. Usage Instructions	10
8. Troubleshooting	10
9. Frequently Asked Questions (FAQs)	10

Part I: Project Development Process

1. Applied Methodology

This project utilized the **Agile Scrum** methodology. This approach was chosen to allow for iterative development, frequent feedback loops, and flexibility in managing requirements.

2. Planning and Risk Analysis

The project timeline is divided into 8 sprints, targeting a final release date of Wednesday 10.12.2025.

Risk analysis concluded that possible risks during the project were time management, database schema changes and possible life events or getting sick.

All in all, our team managed to handle the risks and the project was delivered on time.

3. Agile Implementation Approach

The development cycle of the project:

- **Sprint Duration:** 2 Weeks
- **CI/CD Pipeline:** Automated testing and analysis were handled using **Jenkins** and **SonarQube** to ensure code quality.
- **Sprint Review:** End of every sprint to demonstrate features.

4. Scrum Team Roles and Responsibilities

During the project, scrum master role was changed every sprint.

Project Links:

- **Source Code:** <https://github.com/samuelms123/OTP-1>
- **Trello:** <https://trello.com/w/1otp1/home>

Roles:

- **Auri Laitinen:** Full Stack Developer/Scrum Master
- **Niko Mehiläinen:** Full Stack Developer/Scrum Master
- **Samuel Sarimo:** Full Stack Developer/Scrum Master

Part II: Product Documentation

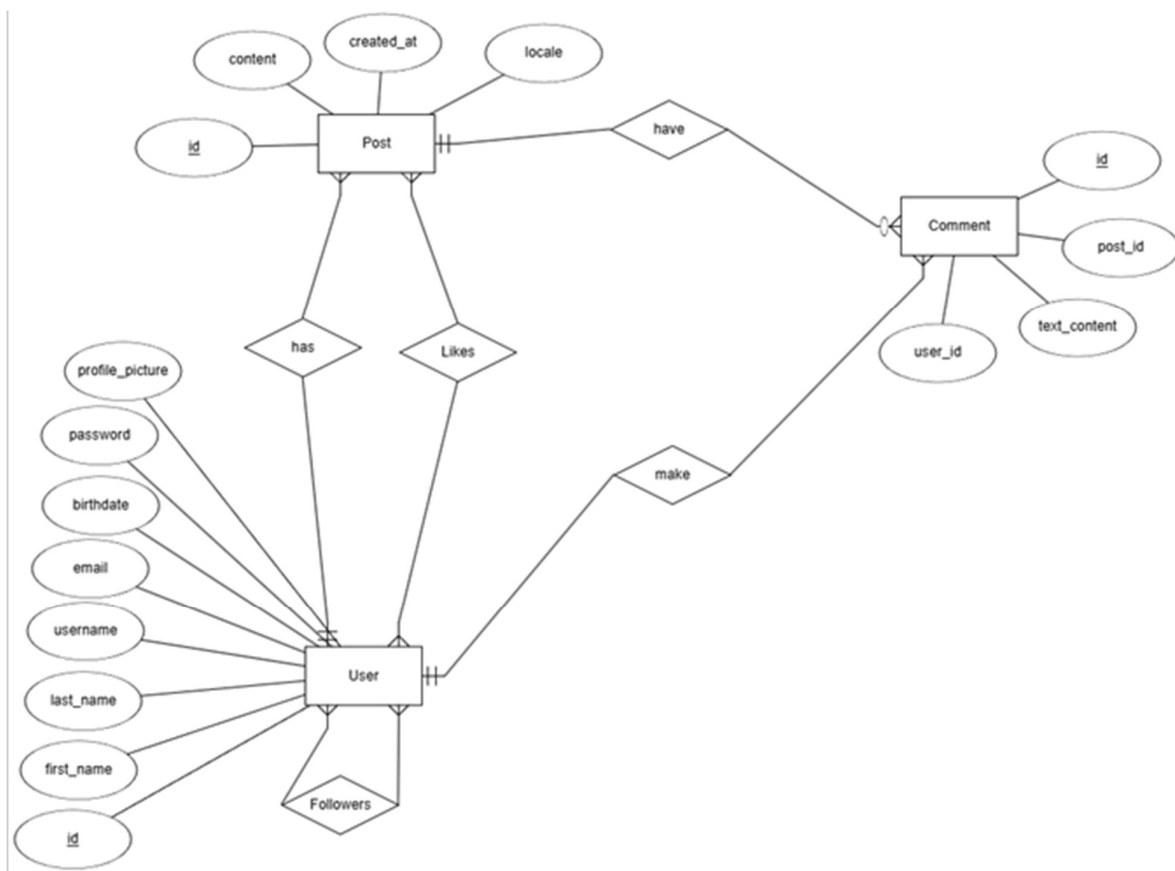
1. Introduction

- **Purpose:** Solve the problem of social media feeling impersonal and algorithm-driven, instead of authentic and meaningful.
- **Scope:** User authentication, CRUD operations, profile modification, local database for storing user data, clean UI with multi-language support.
- **Target audience:** Users seeking private social networking experience. Friend groups for example.

2. Design

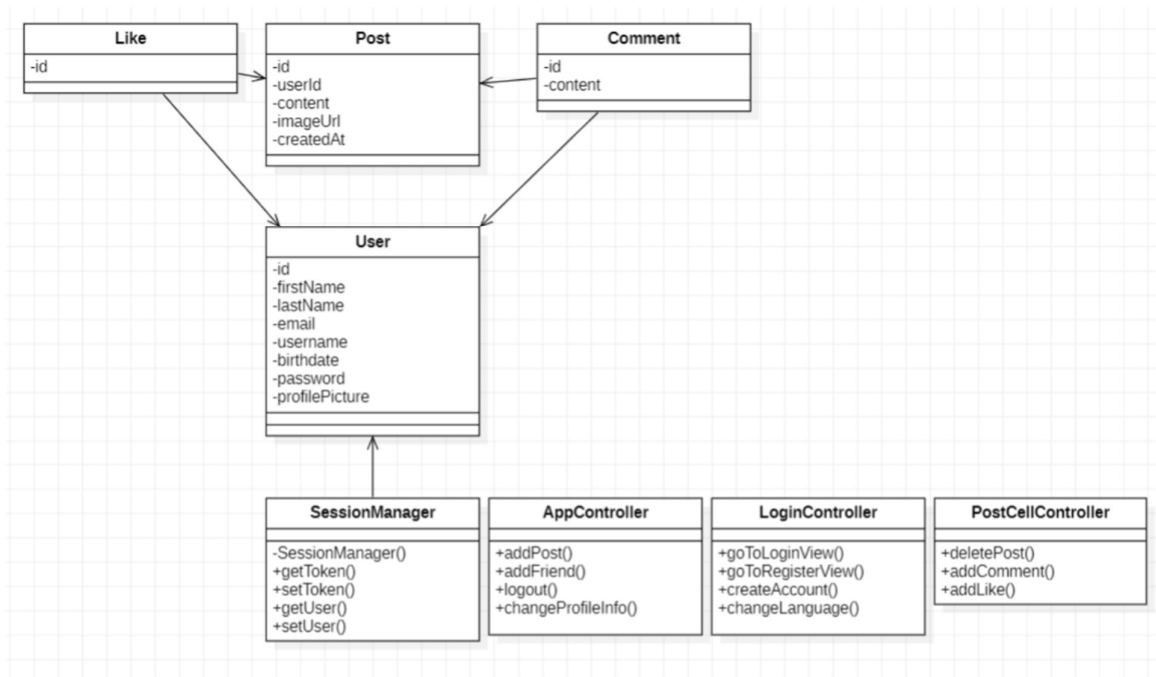
2.1 ER Diagrams

The ER diagram was very simple – it consisted only 3 entity types: Post, Comment and User.

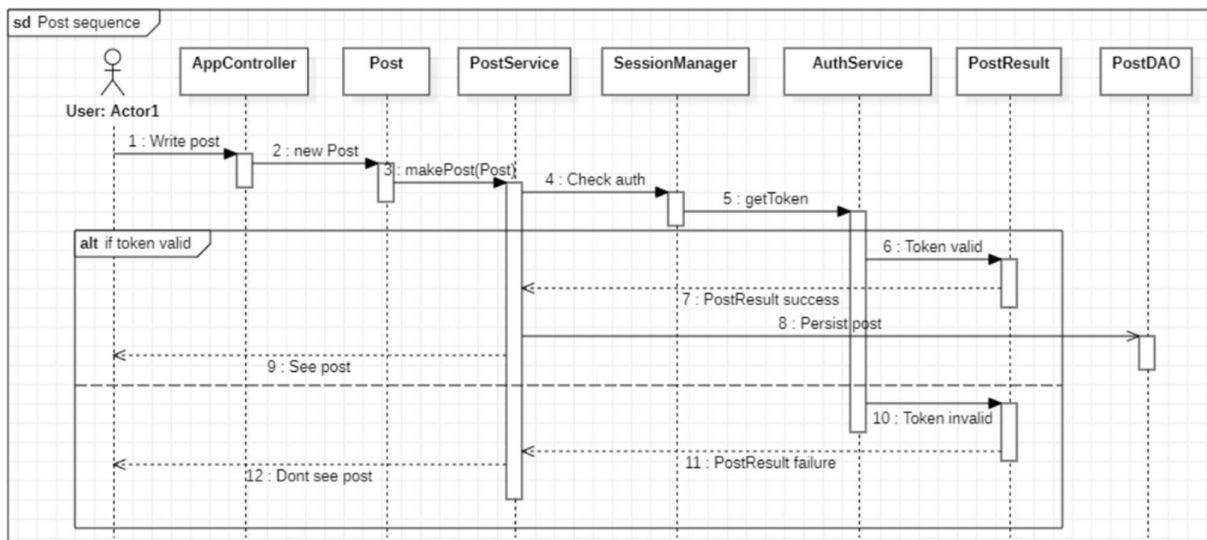


2.2 UML Diagrams

Class Diagram



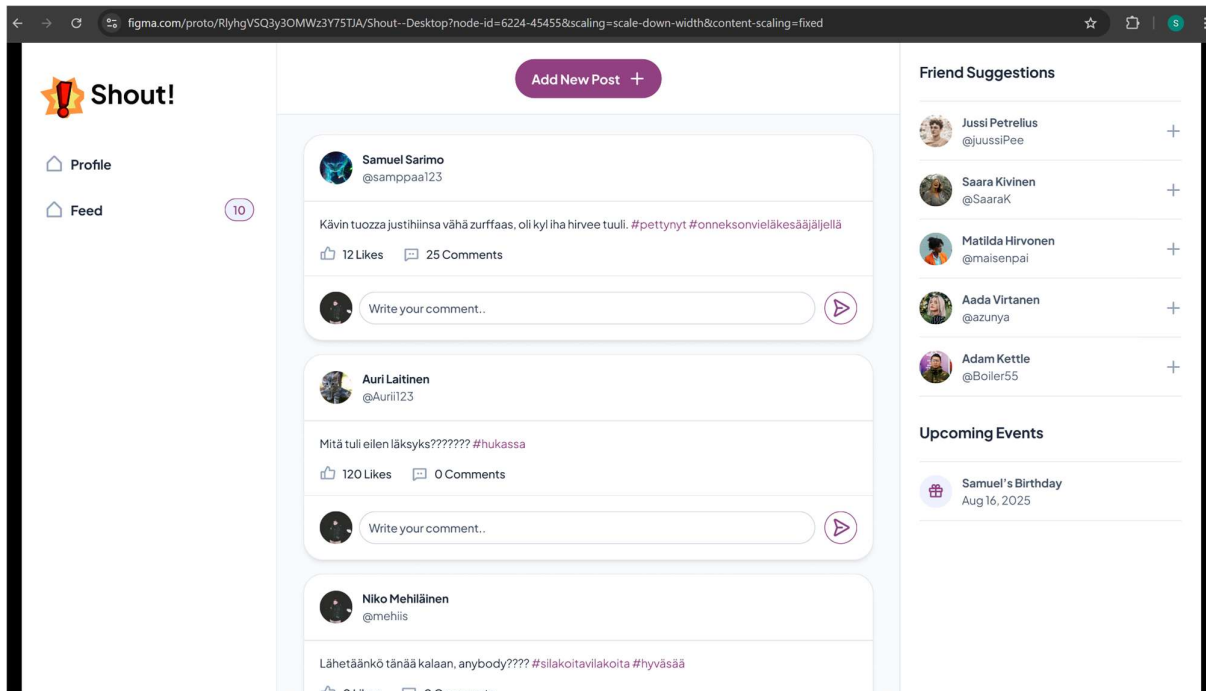
Sequence diagram (Posting)



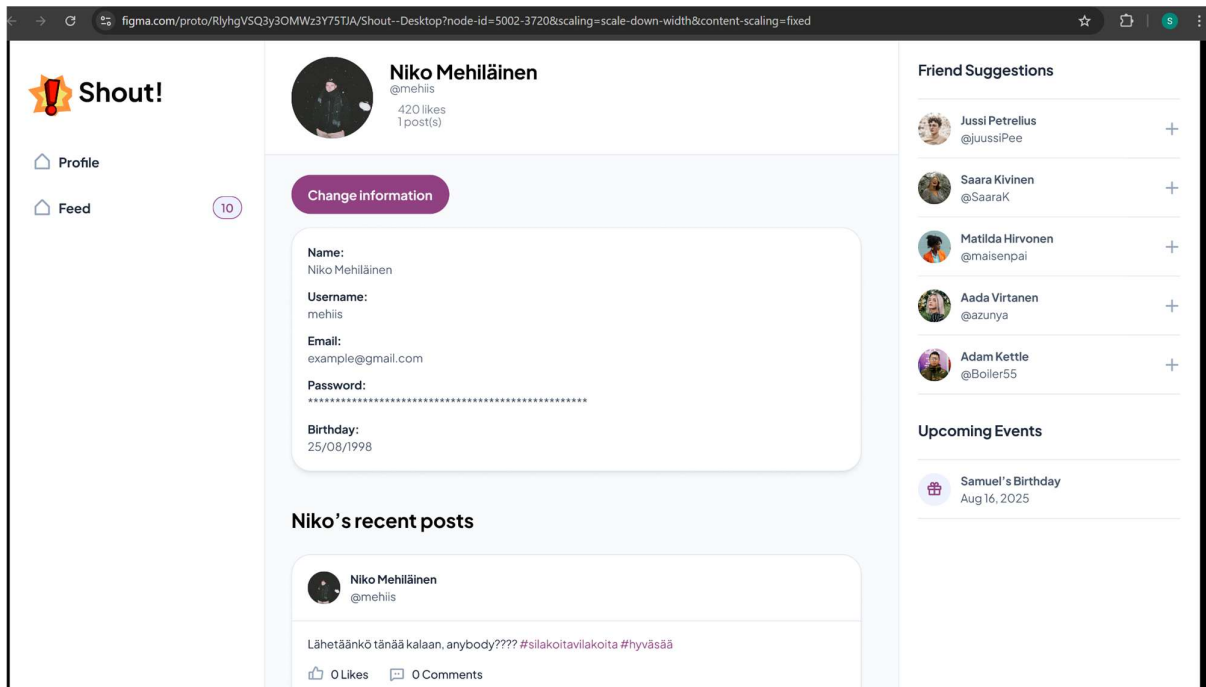
2.3 UI Mockups

UI mockups done with **Figma**:

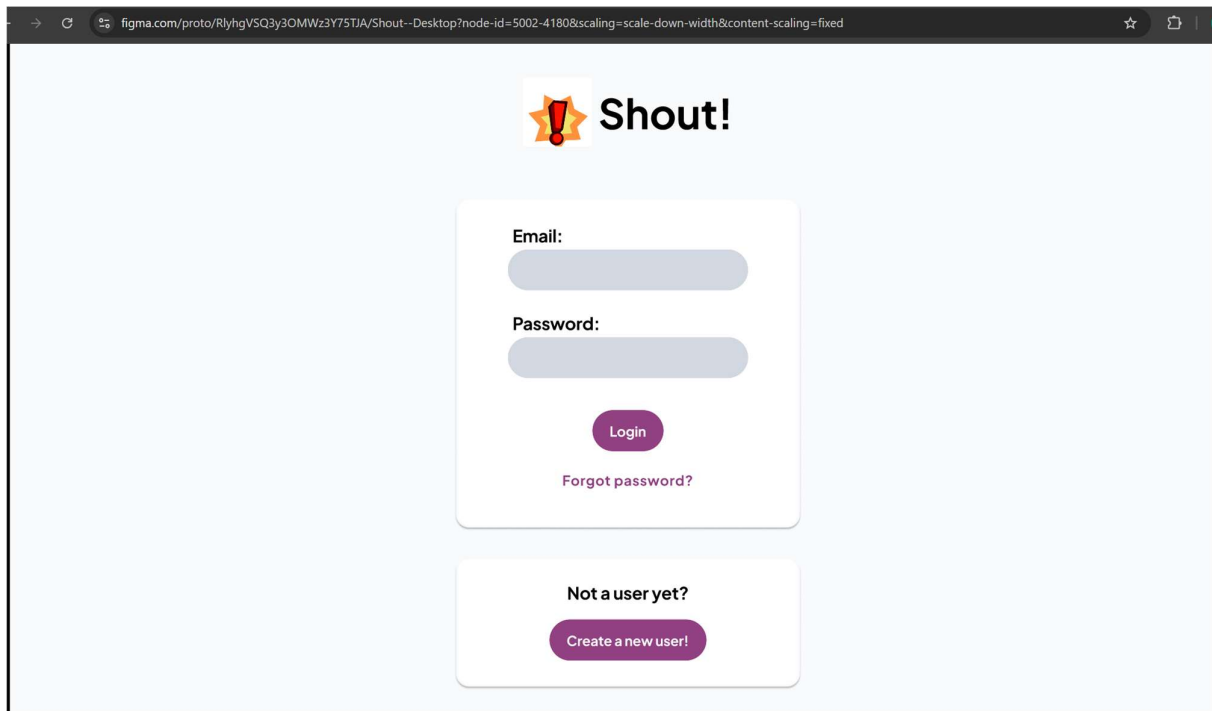
FEED PAGE



PROFILE PAGE



LOGIN/REGISTER



3. Features Overview

- **User Authentication:** Secure login and registration using JWT and BCrypt.
- **Basic Social Media Operations:** Post, like, comment, add/remove friend, modify profile.
- **UI-localization:** Finnish, English, Japanese, Farsi.
- **Database-localization:** Finnish, English, Japanese, Farsi.

4. Implementation Details

- **Frontend:** Java, JavaFX, Scene Builder
- **Backend:** Java, Hibernate (Jakarta Persistence API)
- **Database:** MariaDB
- **Security:** BCrypt (hashing), JWT (tokens)
- **CI/CD:** Docker, Jenkins
- **Testing:** Junit
- **Static Code Analysis:** Sonarqube, PMD, SpotBugs
- **Reporting:** Jacoco

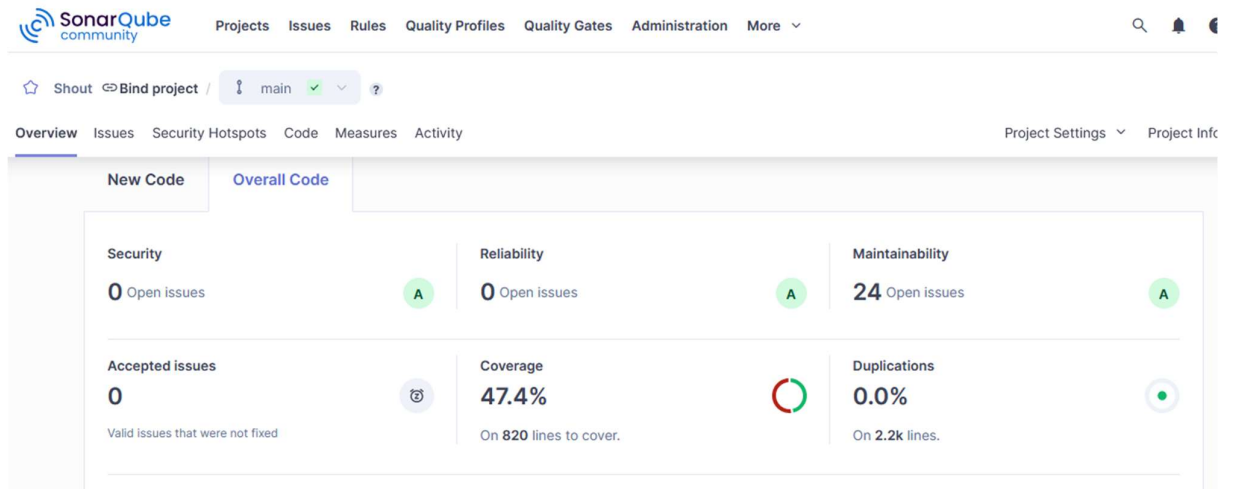
5. Testing Strategy and Results

5.1 Testing Strategy

- **Unit & Integration Testing (JUnit):** We used JUnit to test individual backend components and database interactions.
- **Performance Testing (Apache JMeter):** JMeter was utilized to simulate load on the backend.
- **User Acceptance Testing (UAT):** End-to-end scenarios were validated to ensure the software met the business requirements.
- **Usability Evaluation (Nielsen's 10 Heuristics):** A manual usability inspection was done by the team based on Jakob Nielsen's 10 Usability Heuristics.
- **Constant Static code analysis:** Static code analysis was mainly done with SonarQube.

5.2 Test Results

- **JUnit:** All important functionalities tested with almost 50% coverage. (monorepo)
- **JMeter:** Performance testing was done in Jenkins pipeline and the results were successful.
- **Static Code Analysis:** Codebase maintained stable and clean throughout the project.



- **Usability Findings:**

No	Heuristic	Description of The Issue	Screenshot	Severity	Suggested Improvement
1	H1-1: Simple & Natural dialog	No issues.			
2	H1-2: Speak the users' language	No issues.			
3	H1-3: Minimize users' memory load	In log in screen, there is only small arrow for users to change preferred UI language. User must 'remember' that the arrow is for language settings.		2	Make clear indicator that this is where you change language for the UI - for example placeholder text 'Language'.
4	H1-4: Consistency	Some of the text not visible on some languages.		3	Make text size dynamic.
5	H1-5: Feedback	Program state is not communicated to the user clearly.		2	Make loading bars, success message pop ups.
6	H1-6: Clearly marked exits	Log out button might be unrecognizable for some users.		2	Add 'Log Out' text with the icon.
7	H1-7: Shortcuts	Lacks enter shortcut for log in.		2	Add event listener for enter to log in.
8	H1-8: Precise & Constructive error messages	There is a problem in the UI when there is no connection to database. User is not informed about the issue.		3	Add error prompt for users when database is down and log out.
9	H1-9: Prevent errors	User is allowed to change their email to whatever they want without any email validation. Leading in possible future errors.	Email: jeeTääOnMunSähkis	4	Add email validation when user is changing their email.
10	H1-10: Help and Documentation	No instructions on how to use the app.		2	Add guidance tooltips to UI elements.

6. Installation Guide

1. Clone the repository: git clone <https://github.com/samuels123/OTP-1>
2. Change directory: cd OTP-1
3. Copy the env file: cp .env.example .env
4. Fill in the .env file SALT_ROUNDS=5 and JWT_SECRET=DSIASIDUAHSDIHA3 or whatever you like
5. Create new MariaDB database, script found in the **docs** -folder of the project
6. Start the program in the root directory: mvn javafx:run

Dependencies are installed and the software is executed.

7. Usage Instructions

- **Step 1:** Create a new profile in the register section
- **Step 2:** Log in with the new user.
- **Step 3:** Start using **Shout!**

8. Troubleshooting

- Problem: Registration/Login fails due faulty database connection.
 - Check if you have correctly followed the *sql_script -file in the documents folder*.

9. Frequently Asked Questions (FAQs)

- Q: How do I delete my account?
- A: At the moment you **cannot**, soon to be implemented.

- Q: Does Shout! support multiple languages?
- A: Yes, Shout! supports English, Finnish, Japanese and Farsi.

10. Support and Contact Information

For technical support, please contact support@shout.fi .